ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP E1–E3

CO5085 · Học sâu và ứng dụng trong thị giác máy tính

Học kỳ 261 · Năm học 2026–2027

Phân loại ảnh với CNN, attention và mạng hồi tiếp

Học viên

Huỳnh Thành Đạt	2570161
Giảng viên	Lê Thành Sách
	Nguyễn Quốc Minh

Thành phố Hồ Chí Minh, tháng 11 năm 2026

Tóm tắt

Ba bài tập khảo sát cách biểu diễn ảnh trong bài toán phân loại Fashion-MNIST. E1 so sánh bộ phân loại softmax, MLP và CNN. E2 đối chiếu multi-head self-attention tự viết với mô-đun PyTorch trên ba cách tạo token. E3 so sánh LSTM và GRU khi ảnh được đọc theo hàng hoặc theo patch, đồng thời khảo sát độ nhạy với thứ tự quét.

Mỗi cấu hình được huấn luyện từ đầu với ba seed. CNN đạt độ chính xác trên tập test trung bình cao nhất, 92,69%. Trong nhóm attention, bản tự viết dùng CNN-stem đạt 88,59%. GRU đọc ảnh theo hàng đạt 90,07%. Phân tích gradient và phép thay đổi thứ tự quét mô tả độ nhạy của mô hình đối với biểu diễn chuỗi.

Tổng quan báo cáo

Dữ liệu	Fashion-MNIST, 10 lớp. Ba bài dùng cùng các tập train, validation và test.
E1	So sánh softmax, MLP và CNN về độ chính xác, số tham số và lỗi phân loại.
E2	Đối chiếu hai hiện thực attention trên token theo hàng, theo patch và theo đặc trưng CNN-stem.
E3	So sánh LSTM, GRU với MLP và CNN. Khảo sát độ nhạy với thứ tự quét và gradient theo bước.
Phép đo	13 cấu hình, ba seed mỗi cấu hình. Checkpoint được chọn trên tập validation.
Kết quả chính	CNN đạt 92,69% độ chính xác trên tập test trung bình. GRU đọc theo hàng đạt 90,07% trong E3.

Mục lục

Tóm tắt **i**

Tổng quan báo cáo **ii**

1 Dữ liệu và quy trình chung **1**

1.1 Phạm vi bài làm 1

1.2 Tập dữ liệu và cách chia 1

1.3 Huấn luyện và chỉ số đánh giá 2

2 E1: Softmax, MLP và CNN **3**

2.1 Ba bộ phân loại 3

2.2 Kết quả và phân tích lỗi 4

3 E2: MSA và cách lấy token **7**

3.1 Hiện thực attention 7

3.2 Ba cách tokenization 7

3.3 So sánh attention và tokenization 9

4 E3: LSTM/GRU trên chuỗi ảnh **12**

4.1 Biểu diễn ảnh và kiến trúc 12

4.2 Các phương trình được sử dụng 13

4.3 Kết quả và quan hệ không gian 13

4.4 Độ nhạy với thứ tự quét 16

5 Phạm vi thực nghiệm và giới hạn **18**

5.1 Cấu hình và các lượt chạy 18

5.2 Kết luận 18

Tài liệu tham khảo **19**

Danh sách hình vẽ

2.1 Ba kiến trúc E1. Kích thước tensor không gồm batch; k là kernel size và p là padding. CNN có hai block convolution trong cùng một mô hình. Các đầu ra đều là logits. 3

2.2 Validation CE của ba mô hình E1 ở seed 42. 5

2.3 Validation accuracy của E1; seed dùng cho hình ghi trên biểu đồ. 5

2.4 Confusion matrix test của CNN ở seed đại diện. 6

2.5 Một số dự đoán sai có confidence cao của CNN. 6

3.1 MSA tự viết với bốn head, mỗi head 16 chiều. Các thao tác Linear, reshape, matmul, softmax và ghép head tương ứng trực tiếp với hiện thực; $N = 28$ hoặc 49 tùy tokenizer. Kích thước không gồm batch. 8

3.2 Ba tokenizer E2 đều tạo embedding 64 chiều. Token theo hàng và patch dùng Linear; CNN-stem tạo đặc trưng qua hai convolution stride 2. Kích thước không gồm batch. 8

3.3 Classifier E2 và chi tiết encoder block. Các nhánh residual giữ nguyên kích thước $N \times 64$. Dropout sau MSA trong sơ đồ là dropout ngoài nhánh attention; dropout của attention weights được thể hiện ở Hình 3.1. . . . 9

3.4 Validation CE của sáu cấu hình E2 ở seed 42. 11

3.5 Validation accuracy khi thay attention backend và tokenizer. 11

4.1 Bốn cấu hình E3 dùng cùng sơ đồ xử lý, thay loại cell và cách đọc ảnh. Token đầu vào không qua embedding; mũi tên giữa các cell truyền hidden state và, với LSTM, cả cell state. Chỉ hidden state ở bước cuối được đưa vào classifier. 12

4.2 Cross-entropy trên tập validation của các mạng hồi tiếp và baseline E1 ở cùng seed. 14

4.3 Độ chính xác trên tập validation của E3, MLP và CNN. 15

4.4 Chuẩn gradient theo bước của LSTM đọc ảnh theo hàng trên tập validation ở seed 42. 16

Danh sách bảng

1.1 Thiết lập huấn luyện dùng chung cho E1–E3. 2

2.1 Kiến trúc E1. Conv không dùng bias vì sau đó có batch normalization. . 4
2.2 Kết quả E1 qua ba seed 42, 43 và 44. 4

3.1 Tokenization của E2 trên ảnh $1 \times 28 \times 28$. 8
3.2 Sáu cấu hình E2. Manual và PyTorch của cùng tokenizer có cùng số tham số. 10

4.1 Kết quả E3 và hai baseline E1 trên cùng phân hoạch dữ liệu. 14

5.1 Các cấu hình được huấn luyện và đánh giá với ba seed 42, 43 và 44. . . . 18

Chương 1

Dữ liệu và quy trình chung

1.1 Phạm vi bài làm

Báo cáo so sánh các mô hình phân loại ảnh trên cùng tập Fashion-MNIST. E1 gồm softmax, MLP và CNN. E2 trình bày hiện thực multi-head self-attention (MSA), đối chiếu với thư viện và khảo sát ba cách tạo token. E3 biểu diễn ảnh thành chuỗi để phân loại bằng LSTM/GRU, với MLP và CNN của E1 làm baseline. Chỉ số bước của mạng hồi tiếp tương ứng với thứ tự quét các vùng trên ảnh.

Báo cáo tổng hợp 13 cấu hình, gồm ba cấu hình E1, sáu cấu hình E2 và bốn cấu hình hồi tiếp E3. Mỗi cấu hình được huấn luyện từ đầu với ba seed 42, 43 và 44, tương ứng 39 lượt. Các mô hình dùng chung cách chia dữ liệu và quy tắc chọn checkpoint.

1.2 Tập dữ liệu và cách chia

Fashion-MNIST được lấy từ kho dữ liệu chính thức của Zalando Research [2]. Bộ dữ liệu gồm 60.000 ảnh thuộc tập train và 10.000 ảnh thuộc tập test, mỗi ảnh có kích thước 28×28, một kênh grayscale và thuộc một trong 10 lớp sản phẩm thời trang. Ảnh và nhãn được cung cấp dưới dạng các tệp IDX nén gzip. Kích thước ảnh nhỏ phù hợp để khảo sát cả CNN, attention và mạng hồi tiếp trên cùng đầu vào.

Từ 60.000 ảnh train, tách phân tầng 10% làm validation với split seed 2026. Như vậy có 54.000 ảnh train, 6.000 validation và 10.000 test. Mỗi lớp lần lượt có 5.400, 600 và 1.000 ảnh theo phân bố chuẩn của bộ dữ liệu. Split seed không đổi giữa E1–E3 và không trùng vai trò với seed khởi tạo mô hình.

Ảnh được chuyển từ uint8 sang $[0, 1]$, sau đó áp dụng normalization $x' = (x - \mu)/\sigma$. Hai thống kê μ, σ được tính trên 54.000 ảnh train và dùng lại cho validation/test. Các thí nghiệm không dùng augmentation để giữ đầu vào thống nhất khi so sánh cách biểu diễn ảnh.

Tập test không tham gia chọn checkpoint hay tính thống kê normalization.

1.3 Huấn luyện và chỉ số đánh giá

Mỗi mô hình xuất logits $z \in \mathbb{R}^{10}$. Với nhãn đúng y, cross-entropy là

$$\ell(z, y) = -z_y + \log \sum_{k=1}^{10} \exp(z_k), \qquad \frac{\partial \ell}{\partial z_k} = p_k - \mathbf{1}[k = y]. \tag{1.1}$$

Vì `cross_entropy` đã thực hiện log-softmax ổn định số, đầu vào của hàm loss là logits. Softmax chỉ được dùng khi cần xác suất để trình bày dự đoán.

Thiết lập	Giá trị định trước
Optimizer	AdamW, weight decay 10^{-4} trên các tham số trainable
Ngân sách	30 epoch, batch 256, không early stopping
Learning rate	E1/E3: 10^{-3}; E2: 3×10^{-4}
Scheduler	Cosine decay về 1% learning rate ban đầu
Gradient clipping	Global norm tối đa 1, sau backward và trước step
Độ chính xác số	FP32; tắt AMP, TF32 và compile
Seed mô hình	42, 43, 44; cùng split seed 2026
Chọn checkpoint	Validation accuracy lớn nhất; hòa thì CE nhỏ nhất

Bảng 1.1: Thiết lập huấn luyện dùng chung cho E1–E3.

Vòng lặp huấn luyện gồm zero gradient, forward, tính loss, backward, clipping và optimizer step. Checkpoint được chọn trên tập validation; tập test chỉ được đánh giá sau khi chốt giao thức và hoàn tất huấn luyện các cấu hình đã định. Chương trình kiểm tra tính hữu hạn của loss và gradient tại mỗi bước cập nhật.

Accuracy là chỉ số chính; macro-F1 bổ sung đánh giá theo lớp. Confusion matrix và các dự đoán sai có confidence cao được dùng để phân tích lỗi. Chi phí mô hình được mô tả qua số tham số và thời gian train/epoch. Thời gian được đồng bộ CUDA, gồm lấy batch và cập nhật trọng số; median bỏ epoch đầu để giảm ảnh hưởng khởi tạo. Tốc độ inference được đo riêng, bỏ hai batch đầu và không tính thời gian truyền dữ liệu sang GPU.

Bảng kết quả trình bày độ chính xác trên tập validation trung bình và độ chính xác trên tập test dưới dạng trung bình ± độ lệch chuẩn mẫu qua ba seed. Các đường cong, confusion matrix và ảnh minh họa dùng seed 42. Ngân sách 30 epoch được giữ cố định cho mọi cấu hình; do đó kết quả phản ánh hiệu quả trong ngân sách này.

Chương 2

E1: Softmax, MLP và CNN

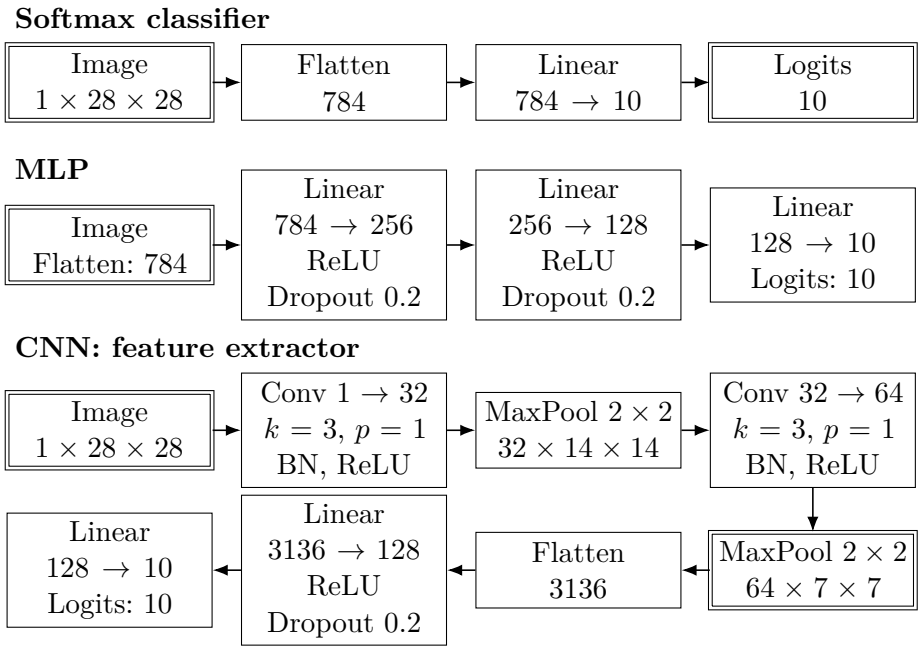
2.1 Ba bộ phân loại

Ba mô hình khác nhau ở cách khai thác cấu trúc ảnh. Bộ phân loại softmax nhận vector $x \in \mathbb{R}^{784}$ và tính logits $z = Wx + b$. Với hai lớp i, j, điều kiện $z_i = z_j$ xác định một siêu phẳng; mô hình không học đặc trưng phi tuyến trước khi phân loại. MLP dùng cùng vector ảnh nhưng bổ sung hai tầng ẩn:

$$h_1 = \mathrm{ReLU}(W_1 x + b_1), \qquad h_2 = \mathrm{ReLU}(W_2 h_1 + b_2), \qquad z = W_3 h_2 + b_3. \tag{2.1}$$

Dropout được áp dụng sau mỗi tầng ẩn khi train. Flatten vẫn giữ thông tin pixel, nhưng các kết nối fully connected không áp đặt quan hệ lân cận hai chiều.

CNN đưa giả định về tính cục bộ và khả năng dùng chung đặc trưng theo vị trí vào kiến trúc. Mỗi kernel 3×3 xử lý một vùng ảnh và dùng chung trọng số khi trượt trên lưới. Padding 1 giữ kích thước không gian qua convolution; max pooling giảm kích thước và giữ đáp ứng mạnh trong từng vùng 2×2. Hình 2.1 trình bày đường truyền dữ liệu và kích thước qua ba mô hình; bảng dưới liệt kê các lớp.



Hình 2.1: Ba kiến trúc E1. Kích thước tensor không gồm batch; k là kernel size và p là padding. CNN có hai block convolution trong cùng một mô hình. Các đầu ra đều là logits.

Mô hình	Các lớp theo thứ tự
Softmax	Flatten $784 \to 10$
MLP	Flatten; $784 \to 256$; ReLU; dropout 0,2; $256 \to 128$; ReLU; dropout 0,2; $128 \to 10$
CNN	Conv $1 \to 32$, kernel 3, padding 1; BN; ReLU; pool 2; Conv $32 \to 64$ tương tự; pool 2; flatten; $3136 \to 128$; ReLU; dropout 0,2; $128 \to 10$

Bảng 2.1: Kiến trúc E1. Conv không dùng bias vì sau đó có batch normalization.

Hai phép pooling đưa ảnh $28 \to 14 \to 7$, vì vậy head CNN nhận $64 \times 7 \times 7 = 3136$ giá trị. Với một lớp Linear $d_{in} \to d_{out}$ có bias, số tham số là $d_{in}d_{out} + d_{out}$. Ví dụ softmax có $784 \times 10 + 10 = 7.850$ tham số. Các số còn lại được đếm trực tiếp bằng `numel()`, gồm tham số affine của BN nhưng không gồm running statistics.

2.2 Kết quả và phân tích lỗi

Mô hình	Tham số	Val (%)	Test (%)	Giây/epoch
Softmax	7,850	85.98	84.51 ± 0.15	0.28
MLP	235,146	91.02	89.88 ± 0.14	0.36
CNN	421,738	93.05	92.69 ± 0.06	0.47

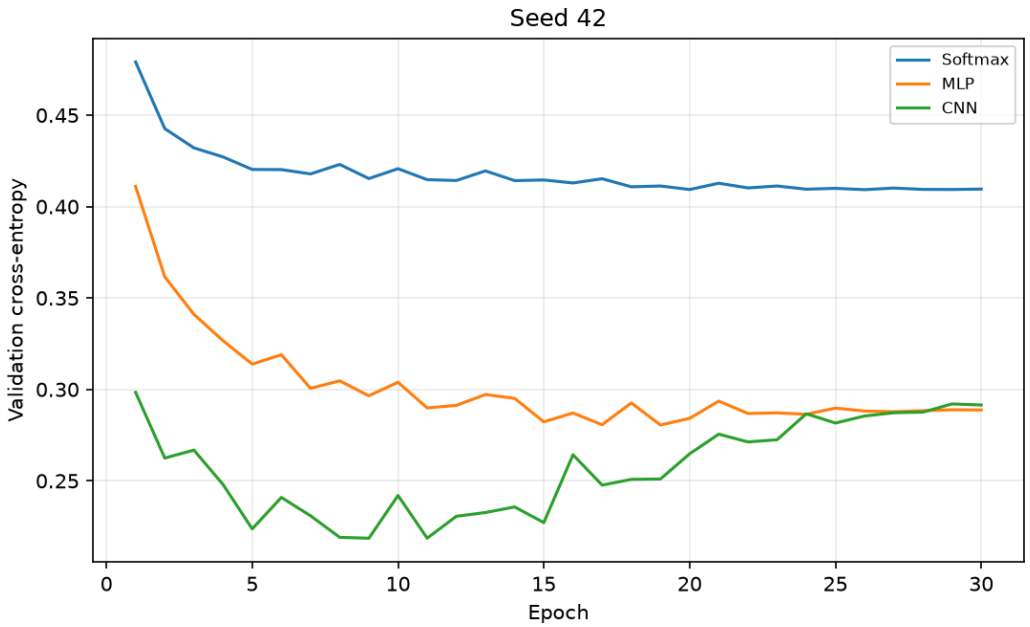
Bảng 2.2: Kết quả E1 qua ba seed 42, 43 và 44.

Val/Test: accuracy (%). Tham số đếm từ kiến trúc; ±: SD qua ba seed. Các lượt chạy được liệt kê ở Chương 5; cách đo thời gian ở Chương 1.

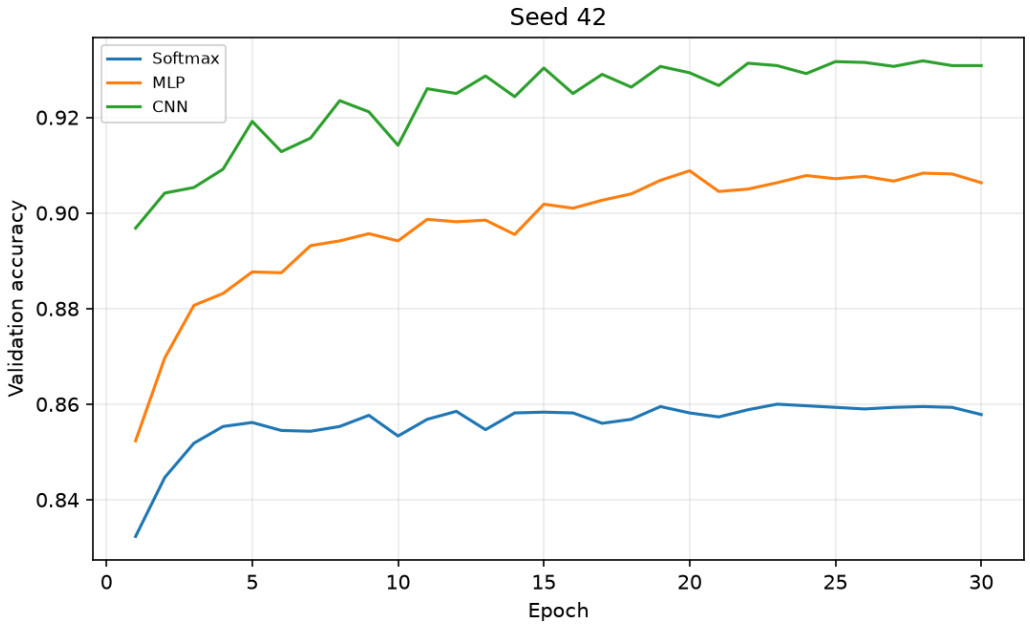
Train/validation CE được đối chiếu với accuracy để phân tích quá trình học. Confusion matrix và ảnh dự đoán sai bên dưới lấy từ CNN ở seed 42. MLP tăng độ chính xác trên tập test từ 84,51% của softmax lên 89,88%. CNN đạt 92,69%, cao hơn MLP 2,81 điểm phần trăm. MLP bổ sung các lớp phi tuyến. CNN còn giữ cấu trúc hai chiều và khai thác lân cận bằng kernel dùng chung. Hai mô hình khác nhau cả về cấu trúc lẫn số tham số.

Ở epoch cuối, cross-entropy của CNN trung bình qua ba seed là 0,0170 trên tập train và 0,2972 trên tập validation. Khoảng cách này cho thấy mô hình khớp tập train nhiều hơn; kết quả test được tính từ checkpoint tốt nhất trên validation, không nhất thiết từ epoch cuối. Trong confusion matrix của seed 42, Shirt có recall 78,8%, thấp nhất trong 10 lớp. Có 86 ảnh Shirt bị gán thành T-shirt/top và 78 ảnh nhầm theo chiều ngược lại. Hai chiều nhầm lẫn này gợi ý khó khăn khi phân biệt kiểu áo ở độ phân giải thấp, nhưng chưa tách được ảnh hưởng của chi tiết ảnh, nhãn và đặc trưng đã học.

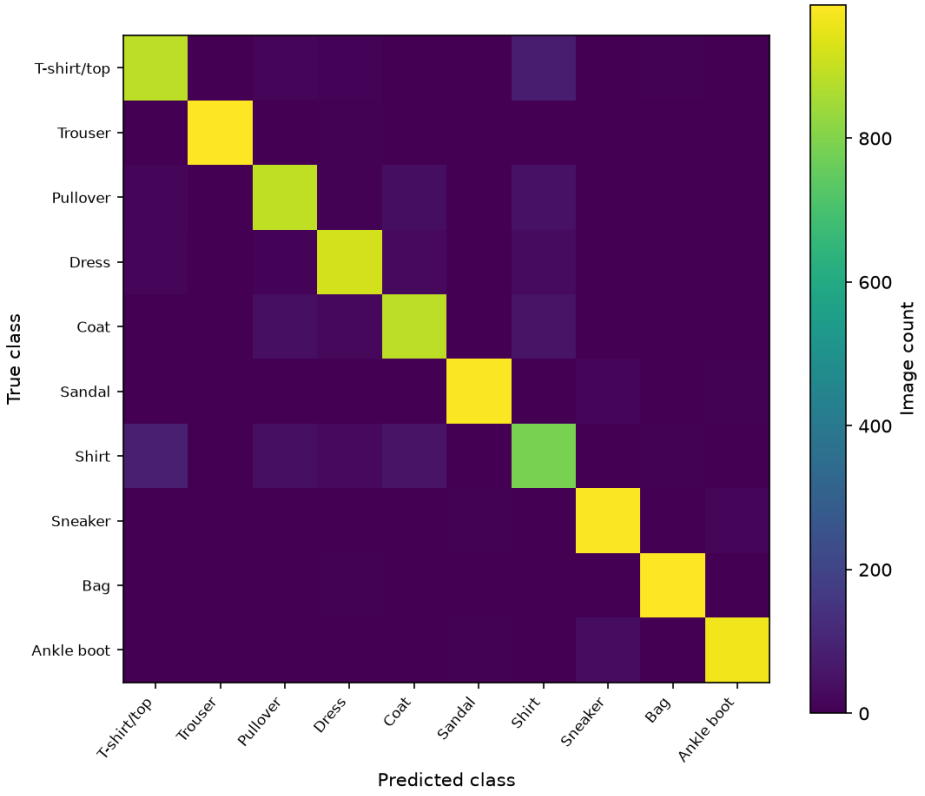
CNN cần khoảng 0,47 giây/epoch, so với 0,36 của MLP và 0,28 của softmax trong cùng môi trường đo. Mức tăng accuracy đi kèm số tham số và chi phí train lớn hơn. Vì kiến trúc và số tham số cùng thay đổi, thí nghiệm chưa xác định riêng mức đóng góp của convolution so với dung lượng mô hình.



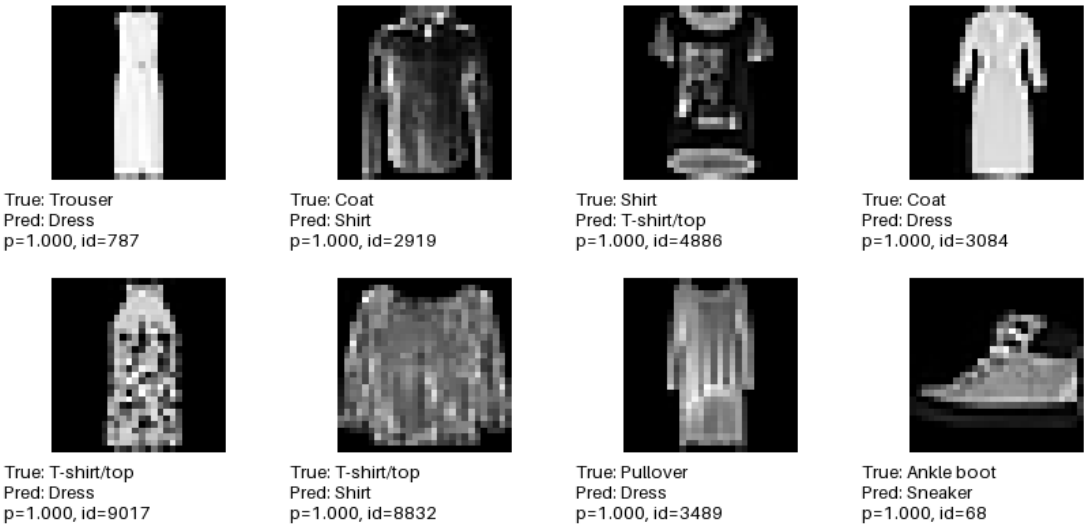
Hình 2.2: Validation CE của ba mô hình E1 ở seed 42.



Hình 2.3: Validation accuracy của E1; seed dùng cho hình ghi trên biểu đồ.



Hình 2.4: Confusion matrix test của CNN ở seed đại diện.



Hình 2.5: Một số dự đoán sai có confidence cao của CNN.

Chương 3

E2: MSA và cách lấy token

3.1 Hiện thực attention

Với tensor token $X \in \mathbb{R}^{B\times N\times d}$, mỗi head học các phép chiếu query, key và value. Bỏ chỉ số batch để gọn, attention của một head là [3]

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \qquad A = \mathrm{softmax}_{\text{hàng}}\left(\frac{QK^\top}{\sqrt{d_h}}\right), \quad O = AV. \tag{3.1}$$

Tích $QK^\top$ biểu diễn mức tương hợp giữa các cặp token. Chia cho $\sqrt{d_h}$ giúp kiểm soát độ lớn của logits khi số chiều tăng; softmax theo chiều key tạo các trọng số có tổng bằng 1 cho mỗi query. Vì $O = AV$, mỗi token đầu ra tổng hợp thông tin từ toàn bộ ảnh thay vì chỉ từ một vùng lân cận. Các head dùng các phép chiếu riêng, sau đó được ghép và chiếu về số chiều ban đầu.

Các bias được lược bỏ trong công thức cho gọn, nhưng có trong hiện thực. Ở đây $d = 64$, có bốn head nên $d_h = 16$. Ba phép Linear được gộp thành QKV, reshape thành các head, nhân ma trận, softmax theo chiều key, áp dụng dropout, ghép head và chiếu output. Bản manual không gọi `nn.MultiheadAttention`, `TransformerEncoder` hoặc fused attention để thay phần tính này.

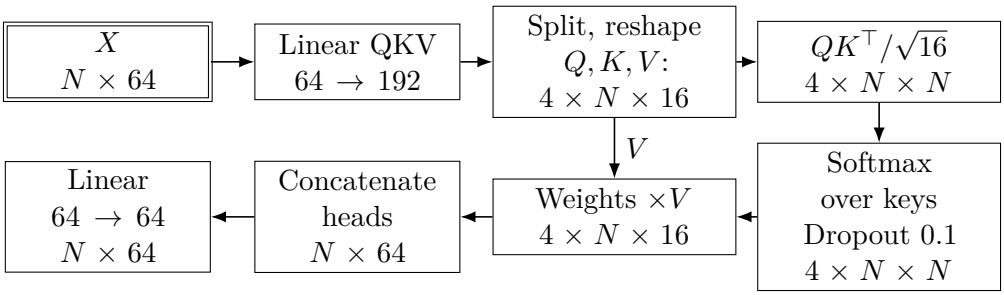
Bản đối chiếu dùng `nn.MultiheadAttention(batch_first=True)`. Các trọng số QKV và output projection của bản tự viết được sao chép sang mô-đun PyTorch. Các phần tokenizer, positional embedding, FFN, LayerNorm và head cũng giữ nguyên tensor khởi tạo của cùng seed.

Khi huấn luyện, dropout và kernel có thể làm hai phiên bản đi theo quỹ đạo khác nhau dù cùng khởi tạo.

Hình 3.1 tách các bước của MSA tự viết. Sau phép QKV chung, tensor được chia thành bốn head; softmax được tính độc lập trên các key của từng head. Phép ghép head đưa tensor về kích thước ban đầu trước output projection.

3.2 Ba cách tokenization

Token hàng gom các pixel cùng một hàng. Token patch quét lưới từ trái sang phải, trên xuống dưới. CNN-stem rút feature map $64 \times 7 \times 7$; mỗi vị trí là một token đã qua xử



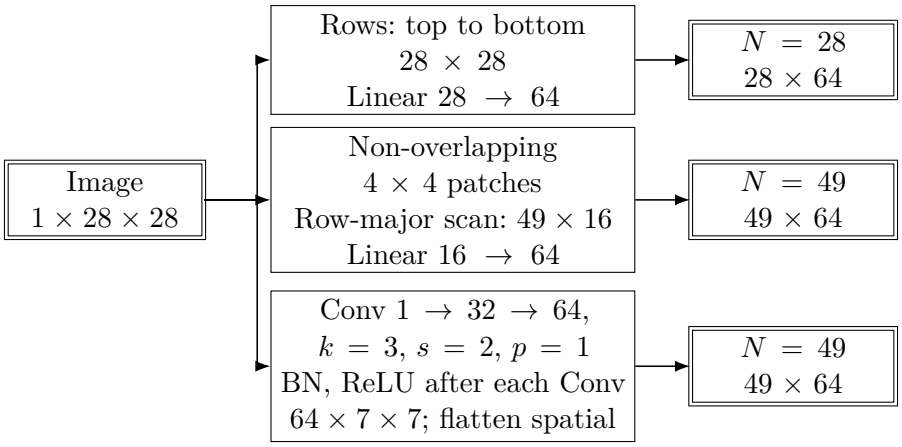
Hình 3.1: MSA tự viết với bốn head, mỗi head 16 chiều. Các thao tác Linear, reshape, matmul, softmax và ghép head tương ứng trực tiếp với hiện thực; $N = 28$ hoặc 49 tùy tokenizer. Kích thước không gồm batch.

Cách chia	N	Đầu vào/token	Phép tạo embedding
Hàng	28	28	Linear $28 \to 64$
Patch 4×4	49	16	Unfold không chồng lấp, Linear $16 \to 64$
CNN-stem	49	64	Hai Conv stride 2: $1 \to 32 \to 64$, BN/ReLU; flatten spatial

Bảng 3.1: Tokenization của E2 trên ảnh $1 \times 28 \times 28$.

lý cục bộ. Stem được huấn luyện từ đầu và bổ sung cả phép tính lẫn tham số so với hai cách tạo token trực tiếp.

Hình 3.2 cho thấy ba cách chuyển cùng một ảnh thành chuỗi. Số chiều embedding đều là 64, trong khi số token phụ thuộc cách biểu diễn.



Hình 3.2: Ba tokenizer E2 đều tạo embedding 64 chiều. Token theo hàng và patch dùng Linear; CNN-stem tạo đặc trưng qua hai convolution stride 2. Kích thước không gồm batch.

Learned positional embedding được cộng vào token để mô hình phân biệt vị trí trên ảnh. Attention đơn thuần không xác định vị trí từ thứ tự lưu token; mean pooling cũng không giữ thứ tự đó. Mỗi cấu hình dùng hai encoder block dạng pre-LN, rồi LayerNorm, mean

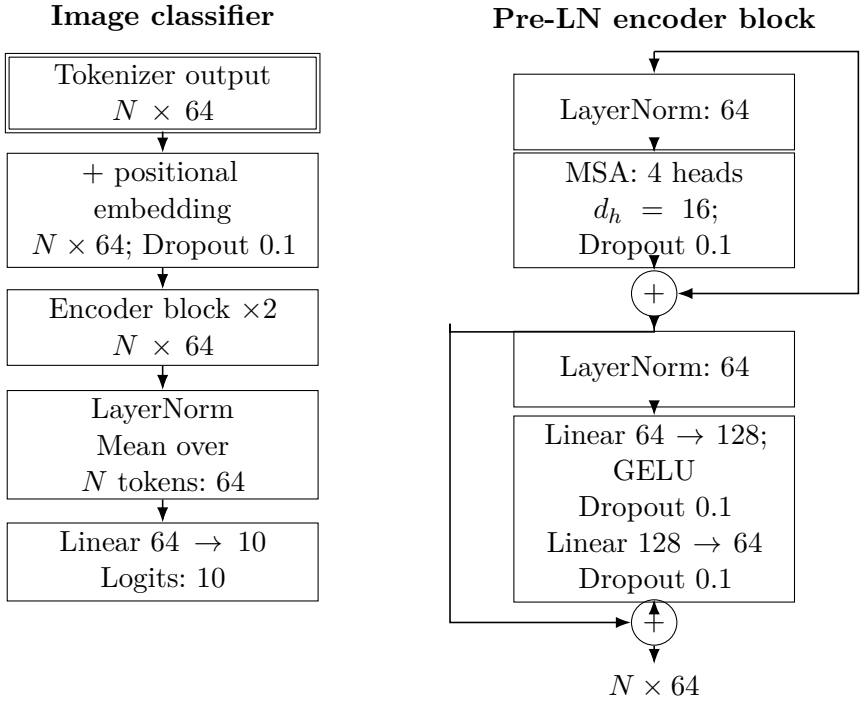
pooling và Linear $64 \to 10$ để phân loại. Không có CLS token. Mỗi block tính

$$U = X + \text{Dropout}(\text{MSA}(\text{LN}(X))), \tag{3.2}$$

$$Y = U + \text{FFN}(\text{LN}(U)), \tag{3.3}$$

trong đó FFN là $64 \to 128 \to 64$ với GELU và dropout 0,1. Residual connection cộng đầu vào vào kết quả của từng nhánh, còn LayerNorm được áp dụng trước attention và FFN theo thiết kế pre-LN. Attention không dùng causal mask vì toàn bộ ảnh đã có tại thời điểm phân loại.

Hình 3.3 đặt encoder block trong classifier hoàn chỉnh. Hai block có cùng cấu trúc nhưng dùng các bộ trọng số riêng. Mean pooling chỉ gộp theo chiều token, giữ lại vector 64 chiều cho lớp phân loại cuối.



Hình 3.3: Classifier E2 và chi tiết encoder block. Các nhánh residual giữ nguyên kích thước $N \times 64$. Dropout sau MSA trong sơ đồ là dropout ngoài nhánh attention; dropout của attention weights được thể hiện ở Hình 3.1.

3.3 So sánh attention và tokenization

Val/Test: accuracy (%). Tham số đếm từ kiến trúc; ±: SD qua ba seed. Các lượt chạy được liệt kê ở Chương 5; cách đo thời gian ở Chương 1.

Manual và thư viện được so sánh ở từng tokenizer; ba tokenizer được so sánh khi giữ attention backend cố định. Chi phí tính điểm attention tỉ lệ $N^2 d$; số cặp token của patch gấp $49^2/28^2 \approx 3,06$ lần token hàng. Thời gian toàn mô hình còn gồm projection, FFN,

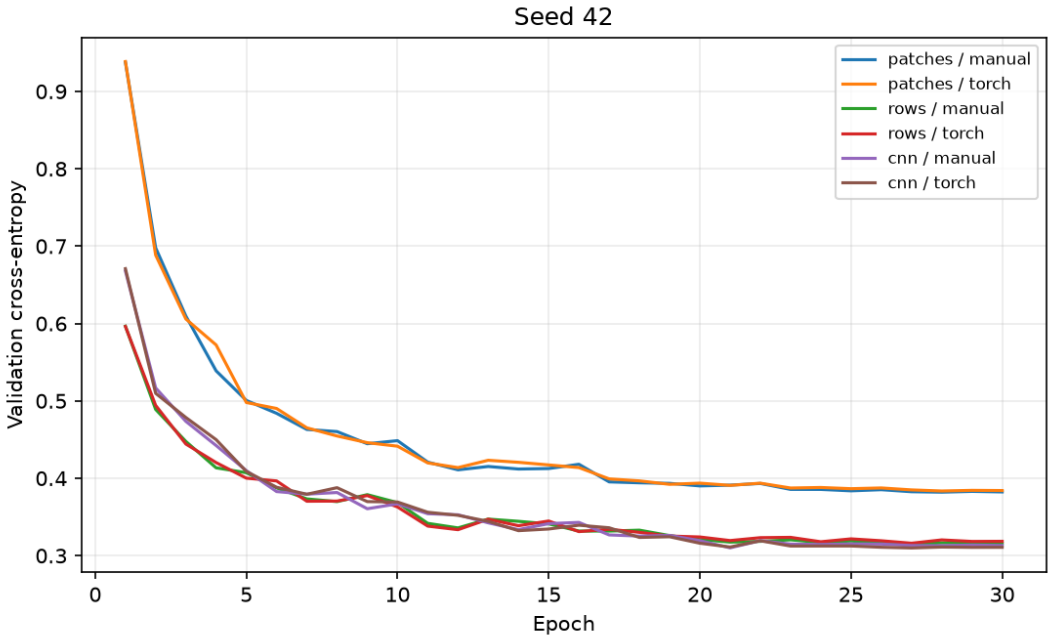
Mô hình	Tham số	Val (%)	Test (%)	Giây/epoch
patches / manual	71,946	87.03	85.93 ± 0.60	1.10
patches / torch	71,946	87.09	86.02 ± 0.37	1.05
rows / manual	71,370	88.61	87.92 ± 0.03	0.99
rows / torch	71,370	88.61	87.86 ± 0.02	0.94
cnn / manual	89,770	89.28	88.59 ± 0.34	1.01
cnn / torch	89,770	89.38	88.50 ± 0.39	0.96

Bảng 3.2: Sáu cấu hình E2. Manual và PyTorch của cùng tokenizer có cùng số tham số.

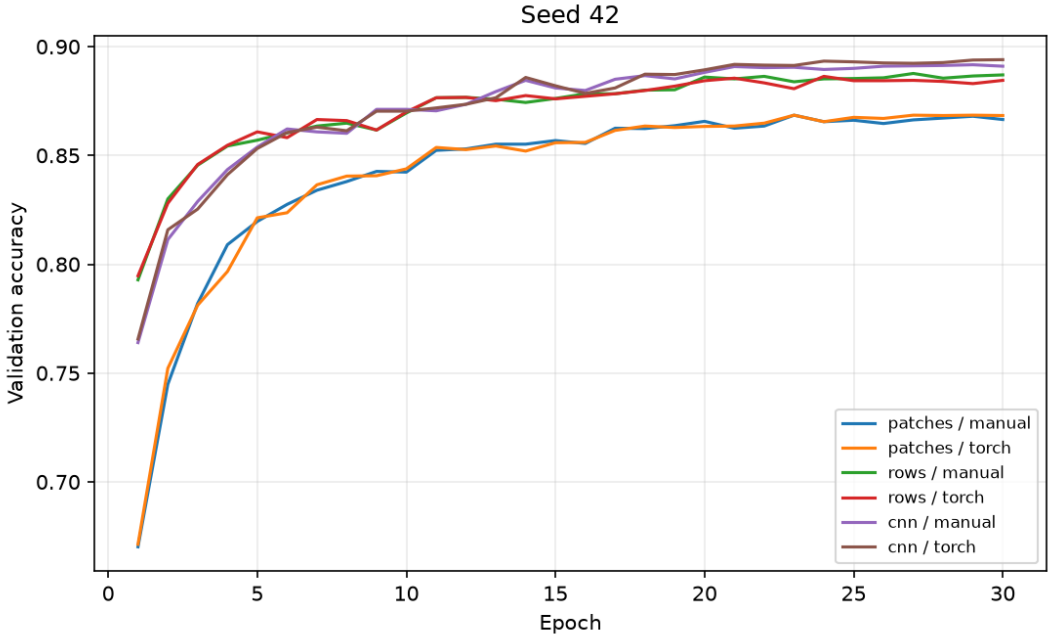
stem và chi phí thực thi kernel.

Trên cùng cách lấy token, chênh lệch độ chính xác trên tập test giữa bản PyTorch và bản tự viết lần lượt là $-0{,}06$, $+0{,}10$ và $-0{,}09$ điểm phần trăm cho token theo hàng, theo patch và theo đặc trưng CNN-stem. Các chênh lệch đều dưới 0,11 điểm phần trăm và không cho thấy một phiên bản luôn có accuracy cao hơn qua cả ba tokenizer. Kết quả này phù hợp với việc hai phiên bản hiện thực cùng phép attention, nhưng ba seed chưa đủ để kết luận tương đương thống kê. Bản PyTorch giảm khoảng 0,05 giây/epoch trong cả ba cặp. Đây là chênh lệch thời gian của toàn bộ classifier trong thiết lập đo, chưa phải phép đo riêng kernel attention.

CNN-stem đạt 88,59% với MSA tự viết và 88,50% với MSA PyTorch, cao hơn token theo hàng và theo patch ở cùng hiện thực. Tầng tích chập đưa đặc trưng cục bộ vào chuỗi trước attention. Cách lấy token đồng thời thay đổi độ dài chuỗi, kích thước đặc trưng và số tham số, nên kết quả phản ánh cả cấu hình biểu diễn. Trong bản manual, CNN-stem cao hơn token hàng 0,67 điểm phần trăm và cao hơn patch 2,66 điểm phần trăm. Tuy vậy, cả ba vẫn thấp hơn MLP và CNN của E1. Kết quả chỉ áp dụng cho các kiến trúc nhỏ được train từ đầu trong 30 epoch; chưa đủ cơ sở suy rộng thành hạn chế chung của attention trong phân loại ảnh.



Hình 3.4: Validation CE của sáu cấu hình E2 ở seed 42.



Hình 3.5: Validation accuracy khi thay attention backend và tokenizer.

Chương 4

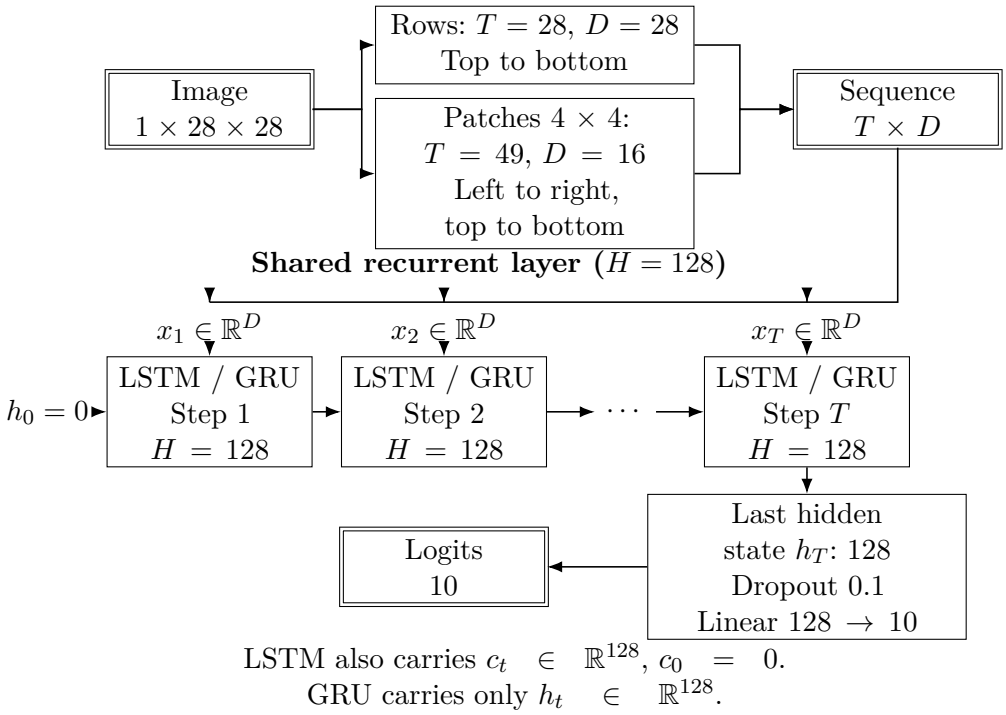
E3: LSTM/GRU trên chuỗi ảnh

4.1 Biểu diễn ảnh và kiến trúc

Ảnh được biến thành chuỗi theo hai cách đã dùng trong E2, nhưng không qua embedding học được: hàng cho $T = 28, D = 28$; patch 4×4 cho $T = 49, D = 16$. Cả LSTM và GRU đều có một tầng, một chiều, hidden size $H = 128$, trạng thái đầu bằng zero. Bộ phân loại lấy hidden state cuối, áp dụng dropout 0,1 rồi chiếu tuyến tính $128 \to 10$.

Quét hàng giữ các quan hệ ngang trong một token và để mạng hồi tiếp xử lý quan hệ giữa các hàng. Quét patch tạo nhiều bước hơn, mỗi bước mô tả một vùng nhỏ hơn. Vì vậy, so sánh này thay đổi đồng thời độ dài chuỗi và kích thước đầu vào tại mỗi bước.

Hình 4.1 biểu diễn mạng khi khai triển theo thứ tự quét. Các bước dùng chung trọng số và hidden size 128; LSTM truyền thêm cell state, còn GRU chỉ truyền hidden state. Các gate điều tiết việc giữ và cập nhật thông tin theo những phương trình bên dưới.



Hình 4.1: Bốn cấu hình E3 dùng cùng sơ đồ xử lý, thay loại cell và cách đọc ảnh. Token đầu vào không qua embedding; mũi tên giữa các cell truyền hidden state và, với LSTM, cả cell state. Chỉ hidden state ở bước cuối được đưa vào classifier.

4.2 Các phương trình được sử dụng

Với LSTM, đặt $a_t = W_x x_t + W_h h_{t-1} + b$ rồi tách thành bốn phần [4]:

$$i_t = \sigma(a_t^i), \qquad f_t = \sigma(a_t^f), \qquad g_t = \tanh(a_t^g), \qquad o_t = \sigma(a_t^o), \tag{4.1}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t). \tag{4.2}$$

c_t là cell state, được cập nhật bằng tổng của phần thông tin cũ giữ lại qua forget gate f_t và phần thông tin mới qua input gate i_t. Output gate o_t điều tiết phần cell state đưa vào hidden state h_t. Trên nhánh cập nhật trực tiếp, đạo hàm theo cell state trước là f_t; qua nhiều bước, nhánh này chứa tích các forget gate. Giá trị gần 1 có thể giúp truyền thông tin và gradient, nhưng toàn bộ đạo hàm còn đi qua các gate và hidden state, nên kiến trúc không tự bảo đảm mọi gradient đều được bảo toàn.

GRU dùng reset gate r_t, update gate z_t và trạng thái ứng viên n_t. Diagnostic khớp đúng cách PyTorch đặt reset gate sau phép chiếu hidden [5]:

$$r_t = \sigma(W_{ir} x_t + b_{ir} + W_{hr} h_{t-1} + b_{hr}), \tag{4.3}$$

$$z_t = \sigma(W_{iz} x_t + b_{iz} + W_{hz} h_{t-1} + b_{hz}), \tag{4.4}$$

$$n_t = \tanh(W_{in} x_t + b_{in} + r_t \odot (W_{hn} h_{t-1} + b_{hn})), \tag{4.5}$$

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}. \tag{4.6}$$

Trong công thức của PyTorch, reset gate tác động sau phép chiếu hidden. Đây là điểm khác với cách viết $W_{hn}(r_t \odot h_{t-1})$. Update gate quyết định mức giữ lại hidden state cũ, còn reset gate điều tiết đóng góp của trạng thái cũ khi tạo n_t. GRU sử dụng một hidden state, còn LSTM tách hidden state và cell state. GRU tính ba nhóm phép chiếu cho reset gate, update gate và trạng thái ứng viên, thay vì bốn nhóm của LSTM. Vì vậy, GRU có ít tham số hơn khi giữ nguyên kích thước đầu vào và hidden size.

PyTorch lưu hai vector bias cho mỗi tầng. Vì vậy, số tham số phần recurrent là $4H(D + H + 2)$ với LSTM và $3H(D + H + 2)$ với GRU; cộng thêm $10H + 10$ cho head. Cùng giá trị H không đồng nghĩa với cùng số tham số giữa hai mô hình.

4.3 Kết quả và quan hệ không gian

Val/Test: accuracy (%). Tham số đếm từ kiến trúc; ±: SD qua ba seed. Các lượt chạy được liệt kê ở Chương 5; cách đo thời gian ở Chương 1.

CNN khai thác lân cận hai chiều và chia sẻ kernel; LSTM/GRU tích lũy thông tin theo đường quét. Khi chuyển từ cuối một hàng patch sang đầu hàng kế tiếp, hai phần tử kề nhau trong chuỗi chưa chắc là hàng xóm gần nhất trên ảnh. Đường quét vì vậy giữ thứ tự một chiều và chỉ bảo toàn một phần quan hệ lân cận hai chiều.

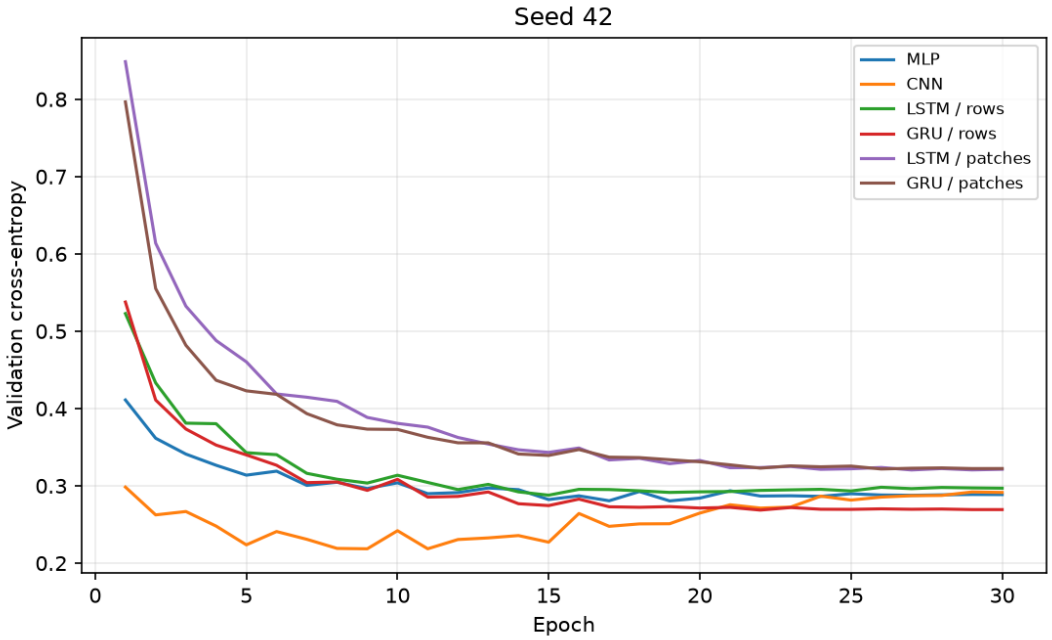
Mô hình	Tham số	Val (%)	Test (%)	Giây/epoch
MLP	235,146	91.02	89.88 ± 0.14	0.36
CNN	421,738	93.05	92.69 ± 0.06	0.47
LSTM / rows	82,186	90.32	89.58 ± 0.05	0.37
GRU / rows	61,962	91.05	90.07 ± 0.17	0.36
LSTM / patches	76,042	88.83	87.65 ± 0.38	0.58
GRU / patches	57,354	88.97	88.00 ± 0.23	0.55

Bảng 4.1: Kết quả E3 và hai baseline E1 trên cùng phân hoạch dữ liệu.

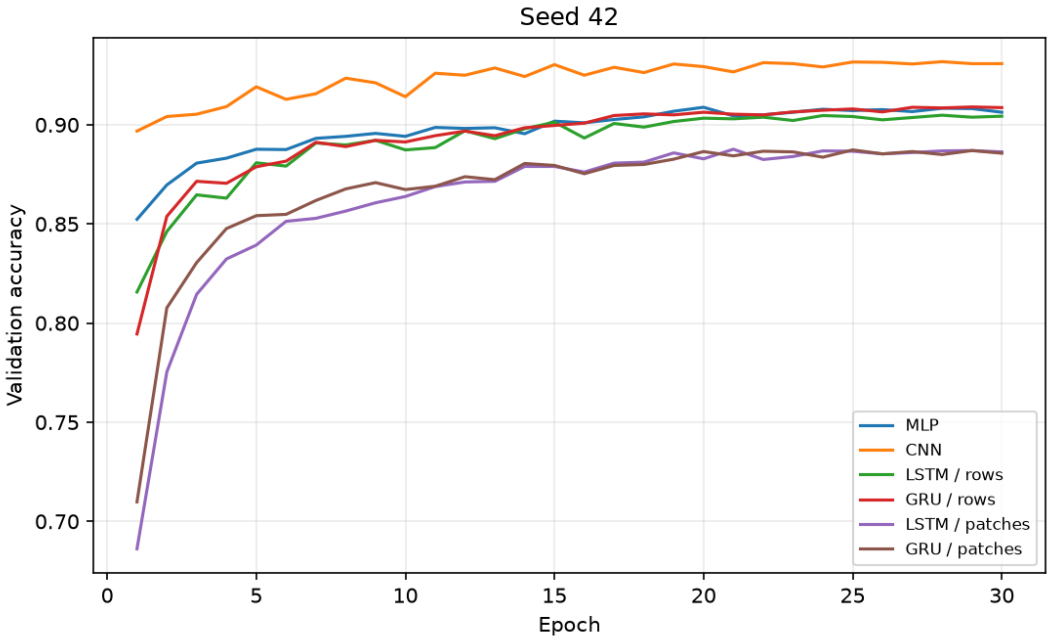
Quét theo hàng tăng độ chính xác trên tập test trung bình so với quét theo patch ở cả LSTM và GRU. Chênh lệch ghép theo seed là $1{,}93 \pm 0{,}42$ điểm phần trăm với LSTM và $2{,}07 \pm 0{,}22$ điểm phần trăm với GRU. Trên cùng biểu diễn theo hàng, GRU cao hơn LSTM $0{,}49 \pm 0{,}21$ điểm phần trăm.

GRU theo hàng đạt 90,07%, gần với MLP (89,88%) và thấp hơn CNN (92,69%). GRU theo hàng có 61.962 tham số, so với 235.146 của MLP và 421.738 của CNN.

GRU theo hàng cần khoảng 0,36 giây/epoch, tương đương MLP trong bảng và thấp hơn GRU theo patch (0,55 giây/epoch). Patch tạo chuỗi 49 bước thay vì 28 bước, dù đầu vào mỗi bước nhỏ hơn. Do các bước hồi tiếp phụ thuộc trạng thái trước, độ dài chuỗi là một yếu tố hợp lý để giải thích khác biệt thời gian; phép đo hiện tại chưa cô lập yếu tố này khỏi kích thước đầu vào và cách thực thi kernel. Tương tự, chênh lệch accuracy giữa hai cách quét phản ánh đồng thời số bước, nội dung token và số tham số.



Hình 4.2: Cross-entropy trên tập validation của các mạng hồi tiếp và baseline E1 ở cùng seed.

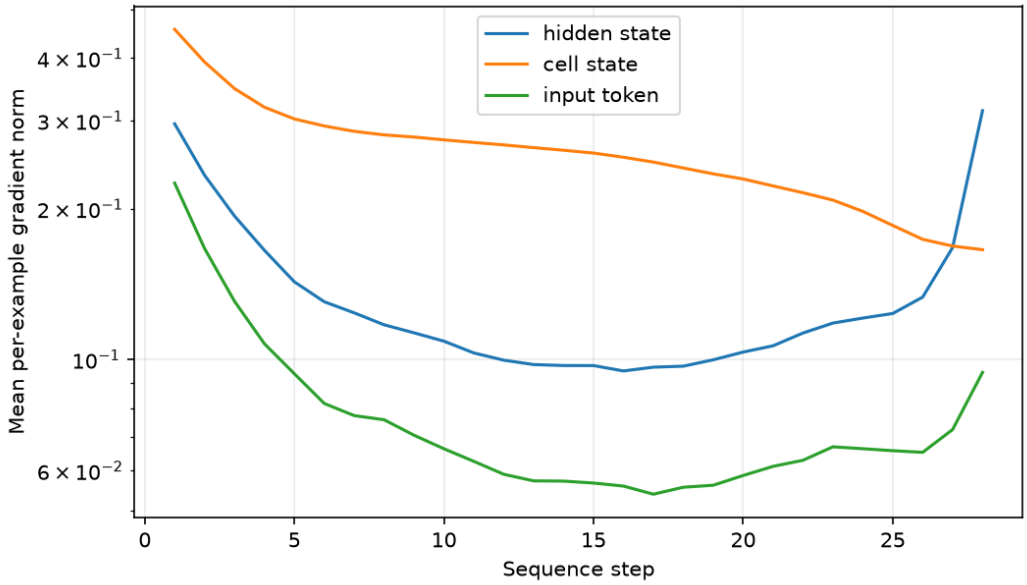


Hình 4.3: Độ chính xác trên tập validation của E3, MLP và CNN.

4.4 Độ nhạy với thứ tự quét

Phép đo dùng checkpoint LSTM đọc theo hàng của mỗi seed và cùng 256 ảnh validation có chỉ số gốc nhỏ nhất. Ba lần đánh giá lần lượt dùng thứ tự gốc, đảo ngược chuỗi và một hoán vị cố định. Trọng số của mô hình giữ nguyên trong cả ba phép đo. Độ chính xác trung bình lần lượt là 89,71%, 20,96% và 44,53%.

Để khảo sát đường truyền gradient, phép khai triển dùng cùng trọng số với mô-đun hồi tiếp và giữ lại đạo hàm của các trạng thái trung gian. Cross-entropy được cộng theo mẫu để tránh hệ số chia theo kích thước batch. Chuẩn gradient được tính cho từng mẫu trước khi lấy trung bình. Hình dưới trình bày gradient theo hidden state, cell state và token đầu vào của LSTM theo hàng ở seed 42.



Hình 4.4: Chuẩn gradient theo bước của LSTM đọc ảnh theo hàng trên tập validation ở seed 42.

Gradient của hidden state và input token giảm từ đầu chuỗi về vùng giữa rồi tăng trở lại ở cuối; gradient của cell state giảm dần nhưng vẫn hiện diện trên toàn bộ 28 bước. Vì classifier sử dụng hidden state cuối, mức tăng gần cuối chuỗi có liên hệ trực tiếp với vị trí tính loss. Đường cong không cho thấy gradient đơn điệu nhỏ dần về đầu chuỗi, nên không thể dùng hình này để khẳng định hiện tượng vanishing gradient. Đây là độ nhạy cục bộ tại checkpoint và nhóm ảnh đang xét; độ lớn gradient cũng không đồng nghĩa với mức đóng góp nhân quả của từng hàng ảnh vào dự đoán.

Mức giảm độ chính xác khi thay đổi thứ tự cho thấy checkpoint nhạy với cách quét đã học. Đảo và hoán vị tạo đầu vào khác phân bố của tập train. Do đó, mức giảm chỉ mô tả độ nhạy trong phép thử này; chưa cho biết mô hình sẽ đạt kết quả ra sao nếu được train lại với thứ tự mới. Việc vẫn phân loại tốt ở thứ tự gốc và giảm mạnh khi đổi thứ tự cho thấy thông tin vị trí trong chuỗi có vai trò đối với checkpoint, nhưng không tự xác nhận

khả năng học phụ thuộc dài hạn. CNN và MLP xử lý trực tiếp lưới pixel hoặc vector cố định, nên đối chiếu với hai mô hình này dựa trên độ chính xác và số tham số. Chuỗi 28 hoặc 49 bước giới hạn phạm vi kết luận về phụ thuộc dài hạn.

Chương 5

Phạm vi thực nghiệm và giới hạn

5.1 Cấu hình và các lượt chạy

Ba notebook đã thực thi 39 lượt huấn luyện và 39 lượt đánh giá trên tập test với RTX 5090, PyTorch 2.10.0 và CUDA 13.0. Bảng và đường cong trong báo cáo lấy từ các lượt này. Bảng dưới mô tả phạm vi kết quả theo từng bài. Hai baseline E1 được dùng lại trong E3.

Bài	Cấu hình	Số cấu hình	Số lượt
E1	Softmax, MLP, CNN	3	9
E2	Hai hiện thực MSA với ba cách tạo token	6	18
E3	LSTM, GRU với hai cách biểu diễn ảnh	4	12

Bảng 5.1: Các cấu hình được huấn luyện và đánh giá với ba seed 42, 43 và 44.

Ba seed cho phép quan sát biến thiên giữa các lượt khởi tạo. Phạm vi so sánh giới hạn ở một tập dữ liệu, ngân sách 30 epoch và các siêu tham số đã chọn. Các chuỗi 28/49 bước cũng ngắn hơn nhiều so với những bài toán phụ thuộc dài hạn.

5.2 Kết luận

CNN đạt độ chính xác trên tập test trung bình cao nhất trong 13 cấu hình, 92,69%. Trong E2, cả hai hiện thực MSA đạt độ chính xác cao nhất với token từ CNN-stem. Trong E3, cả LSTM và GRU đạt kết quả tốt hơn khi đọc ảnh theo hàng. GRU theo hàng đạt 90,07% với số tham số thấp hơn MLP và CNN. Phép thay đổi thứ tự quét trên tập validation cho thấy LSTM theo hàng phụ thuộc vào cách sắp xếp token đã dùng khi huấn luyện.

Phạm vi hỗ trợ của AI được trình bày trong `AI_USE_REPORT.md`.

Tài liệu tham khảo

[1] Lê Thành Sách, Nguyễn Quốc Minh. *Bài tập E1–E3: Học sâu và ứng dụng trong thị giác máy tính, CO5085*. Đề học kỳ 261 (2026–2027).

[2] H. Xiao, K. Rasul, R. Vollgraf. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. arXiv:1708.07747, 2017. https://arxiv.org/abs/1708.07747. Dữ liệu: https://github.com/zalandoresearch/fashion-mnist.

[3] A. Vaswani và cộng sự. *Attention Is All You Need*. NeurIPS, 2017. https://arxiv.org/abs/1706.03762.

[4] PyTorch. *LSTM: API and equations*. https://docs.pytorch.org/docs/2.9/generated/torch.nn.LSTM.html.

[5] PyTorch. *GRU: API and implementation notes*. https://docs.pytorch.org/docs/2.9/generated/torch.nn.GRU.html.

[6] PyTorch. *Previous versions: torch 2.10.0 / torchvision 0.25.0*. https://pytorch.org/get-started/previous-versions/.